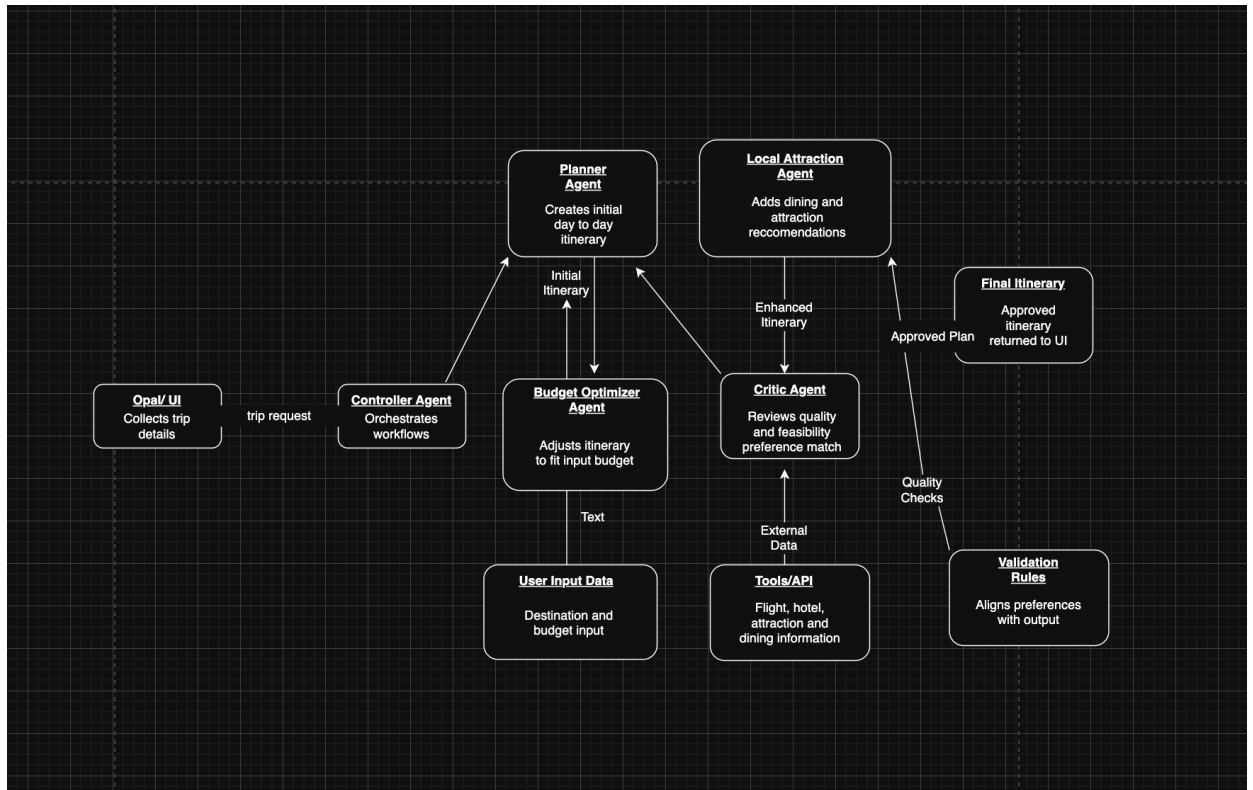


Final Project Milestone II

Agent Architecture Diagram



Agent Roles and Input/Output Formats

The system is composed of multiple agents that are each responsible for a distinct part of the travel planning process. These agents operate sequentially and are coordinated by a central controller.

1. Controller Agent (Orchestrator)

The Controller Agent manages the workflow of the system. It receives user input from the front-end interface and coordinates the execution of all other agents. It ensures that outputs are passed correctly between agents and that the final response is returned to the user.

Input: User-provided data from the UI (destination, budget, duration, preferences)

Output: Final validated travel itinerary

2. *Planner Agent (Task Decomposer)*

The Planner Agent generates the initial travel itinerary. It breaks down the user's request into a structured day-by-day plan based on the destination and preferences.

Input: Destination, Budget, Duration of trip, User preferences

Output:

```
"itinerary": ( "day": 1, "activities": ["Arrival", "City exploration"]),  
"day": 2, "activities": ["Museum visit", "Dining experience"]
```

3. *Budget Optimizer Agent (Analyzer)*

This agent evaluates whether the itinerary fits within the user's budget and modifies activities if necessary to reduce costs.

Input: Initial itinerary from Planner Agent, Budget constraint

Output:

```
"optimized_itinerary": [...],  
"estimated_cost": 1800
```

4. *Local Attraction Agent (Data Enricher)*

The Local Attraction Agent enhances the itinerary by adding relevant attractions, restaurants, and local experiences.

Input: Optimized itinerary, Destination

Output:

```
"enhanced_itinerary":  
"day": 1, "activities": ["Eiffel Tower", "Seine River Cruise"]
```

5. *Critic Agent (Verifier)*

The Critic Agent evaluates the quality, feasibility, and completeness of the itinerary. It determines whether the plan should be accepted or revised.

Input: Enhanced itinerary

Output (Approved):

```
"status": "approved",  
"feedback": "Itinerary is feasible and within budget"
```

Output (Needs Revision):

```
"status": "revise",  
"feedback": "Too many activities per day"
```

Communication Between Agents

Agents communicate through a structured, controller-managed pipeline. The Controller Agent acts as the central hub and ensures that each agent receives the correct input and that outputs are properly formatted before being passed along.

The communication follows a fixed sequence where the Controller receives user input from the front-end interface, the input is passed to the Planner Agent to generate an initial itinerary, the Planner's output is sent to the Budget Optimizer Agent to ensure cost feasibility, the optimized itinerary is then forwarded to the Local Attraction Agent for enrichment, the enriched itinerary is passed to the Critic Agent for validation, and the Critic returns either an approval or a request for revision. If approved, the Controller sends the final result to the user interface.

All data exchanged between agents is formatted as structured JSON-like objects. This format ensures consistency, reduces uncertainty, and allows agents to operate independently without needing direct awareness of agents' internal logic.

If the Critic Agent requests revisions, the Controller sends the feedback back to the Planner Agent, initiating another iteration of the pipeline.

Handling Failures and Low-Quality Outputs

The Controller Agent manages system-level issues such as missing inputs, invalid outputs, or agent failures. It validates inputs before execution, retries failed steps when possible, and provides a simplified secondary itinerary if errors persist. The Critic Agent evaluates the itinerary for feasibility, budget alignment, and relevance to user preferences. If issues are detected, it returns feedback with a "revise" status. The Controller sends this feedback back to the Planner Agent, which updates the itinerary. The pipeline then re-runs until the Critic approves the result or a maximum number of iterations is reached.

Updated Multi-agent Workflow Explained

Since the front-end was built using Opal, the updated multi-agent workflow focuses on system design rather than implementation code. The project was extended by defining a multi-agent workflow where a Controller coordinates several specialized agents (Planner, Budget Optimizer, Local Attraction, and Critic) to generate and refine travel itineraries. Instead of coding these agents, their roles, inputs/outputs, and interactions were clearly structured to simulate how the system would function in a real implementation. The design also introduces a sequential pipeline with a feedback loop, where the Critic agent evaluates the itinerary and requests revisions if needed. This approach demonstrates how the Opal UI can integrate with a scalable, modular multi-agent backend, even though the backend logic is presented conceptually rather than through executable code.

